

Discussion: 12-Factor App applied to ToggleMaster

Challenge goal: read and understand the 12 Factors (12-Factor App) and identify which ones the application already satisfies and which ones would need adjustments for a more robust production environment.

Official reference: <https://12factor.net/>

Summary table

#	Factor	Status in the MVP	Notes
I	Codebase	Yes	One codebase versioned in Git, many deploys (local validation VPS, EC2 planned)
II	Dependencies	Partial	Pinned deps and Docker isolation; base image updated to <code>python:slim</code> (3.14); client pinned to 18 via PGDG; transitive dependencies still lack a lock file
III	Config	Partial	App reads everything from the environment; credentials moved from the compose file to a gitignored <code>.env</code> ; no secrets manager (self-hosted)
IV	Backing services	Yes	PostgreSQL treated as an attached resource (address/credentials via <code>DB_*</code>)
V	Build, release, run	Partial	Docker image separates build/run; deploy is <code>docker compose up --build</code> on the server itself; no versioned releases or rollback; <code>/app</code> bind mount bypasses the image
VI	Processes	Yes	App is stateless (state lives in the database); Gunicorn with workers
VII	Port binding	Yes	App exports HTTP on port 5000; Caddy fronts it with TLS on 80/443
VIII	Concurrency	Partial	Gunicorn runs with 1 worker (no <code>--workers</code>); port publishing blocks horizontal scale-out
IX	Disposability	Partial	Fast startup and graceful SIGTERM via Gunicorn; <code>app</code> service has no healthcheck

#	Factor	Status in the MVP	Notes
X	Dev/prod parity	Partial	Same image local/AWS in theory, but the <code>/app</code> bind mount breaks parity; RDS will introduce divergence
XI	Logs	Partial	Logs go to stdout/stderr as the factor requires; no centralized collection or structured logs
XII	Admin processes	Yes	CLI command <code>flask init-db</code> as a one-off process

Factors satisfied

I. Codebase: the project uses a Git repository with multiple deploys (local validation environment, with EC2 planned).

IV. Backing services: in this environment PostgreSQL is treated as an attached resource. Address and credentials come from the `DB_*` environment variables. Swapping the local container for RDS requires only changing environment variables, zero code.

VI. Processes: in the current state of the application the processes are stateless: no state lives in memory or on disk, everything lives in PostgreSQL. Any request can die without corrupting anything.

VII. Port binding: to expose the API, Gunicorn listens for HTTP on `0.0.0.0:5000`. In my project I also added Caddy to manage TLS and to proxy the web ports 80/443.

XII. Admin processes: Flask's own `init-db` can be considered a one-off process, and it uses the same codebase and environment as the application.

Partially satisfied factors

II. Dependencies: `requirements.txt` declares the technologies with pinned versions and isolation in Docker: every build reconstructs the identical environment from scratch. However, the `python:3.9-slim` base image had been EOL since October 2025, so it was updated to `python:slim` (currently 3.14); the Dockerfile also had an unpinned `apt-get install postgresql-client`, so the client was pinned to major version 18 via the official PGDG repository. In addition, transitive dependencies remain unpinned, without a deterministic lock file. Details in [o6-python-and-dependency-update.md](#).

III. Config: the application already reads everything from the environment (`app.py` uses `DB_*`), and the credentials that were hardcoded in `docker-compose.yaml` have been moved to a gitignored `.env` file that is never sent to the repository. Since we are self-hosting locally there is no Secrets Manager or SSM, so this factor remains partially satisfied. Details in [07-config-via-env-files.md](#).

V. Build, release, run: the Dockerfile separates build from run, but the deployment is done via `docker compose up --build`, meaning build and run happen on the same machine. There is no versioned release and no rollback. To make things worse, the compose file mounts `/app` straight from the machine: the application uses loose files from the server's disk instead of what was included in the image, bypassing the image in "production".

VIII. Concurrency: the process model exists (Gunicorn), but the CMD does not specify `--workers`, so it runs with 1 worker. There is no horizontal scale-out. If we spun up multiple containers for more parallelism, there would be a port 5000 conflict with the current configuration.

IX. Disposability: since the project is still small, startup is fast. The longest wait today is for the database, and SIGTERM is handled properly by Gunicorn. A healthcheck on the `app` service is still missing, so an orchestrator has no way to know when it is ready.

X. Dev/prod parity: in theory we will have the same image running locally and on AWS, but the `/app` bind mount breaks the parity, and the plan to move to RDS introduces divergence.

XI. Logs: the application writes to stdout/stderr, which is exactly what the factor requires. What is missing to complete the factor is centralized collection and structured logs.

Conclusion

Five of the twelve factors are satisfied (I, IV, VI, VII, XII) and seven are partially satisfied (II, III, V, VIII, IX, X, XI); none is fully violated. The fundamentals for cloud readiness (stateless processes, config in the environment, backing services as attached resources) are already in place, and the remaining gaps are mostly at the platform level, to be closed in the AWS phases.

References

- The Twelve-Factor App: <https://12factor.net/>
- [Python and dependency update \(factor II\)](#)
- [Config via environment files \(factor III\)](#)

- Challenge statement: POSTECH - DCLT - Tech Challenge - Fase 1.pdf